

Review of the ETL process for the bookstore

Files:

"Populating Bookstore DW.zip" - Integration Services Project for the Bookstore data warehouse

"sources.zip" - files with source data:

"BillMaster-schema.sql" - creating tables for the computerized sales system

"1.BillMaster.sql" - data for the computerized sales system for the first import (T0-T1)

"2.BillMaster.sql" - data increment for the computerized sales system for the second import (T1-T2)

"1.CEO's.xlsx" – "CEO's excel" for the first import (T0-T1)

"2.CEO's.xlsx" – "CEO's excel" for the second import (T0-T2)

"locations.csv" - auxiliary file containing mapping of postal codes to the provinces

"scripts.zip" - SQL helper scripts:

"bookstoreDW.sql" - creating tables for the data warehouse

"clear_bookstore.sql" – removing data from data warehouse tables

"creating auxiliary tables.sql" - creating temporary tables used in the ETL process

"dropping auxiliary tables.sql" - deleting those tables

"time.sql" - filling the time table in the warehouse

"holidays.sql", "school breaks.sql" - filling in auxiliary tables - dates of holidays and school breaks

Starting the process:

To open the project decompress "Populating Bookstore DW.zip" and open "Populating Bookstore DW.sln" file (VS solution file) in Visual Studio 2019. If SSIS Toolbox does not open along with the solution, open it by selecting "Extensions->SSIS->SSIS Toolbox" (a SSIS package needs to be open to open SSIS Toolbox).

There are two SSIS packages in the project:

"Initial Data.dtsx" - it inserts data into dimensions whose contents are known and there is no reason to extract this data from data sources (in this case:

- the time - it will be 24 entries for each hour,
- date - inserted all the dates of the range used in the sources (1980 -2015), and
- the junk dimension {Junk table} - we assume that the list of all possible positions and forms of payment are known and will not change).

This package is intended to be run only once.

"Recurrent Loading.dtsx" - import of relevant data from data sources to the warehouse. It is intended to be run periodically, although source data has been prepared for only two time points: T1 and T2.

Steps required to run the project:

0. These steps are written with the assumption, that you have created Multidimensional Project for the bookstore organisation and that you have created database bookstoreDW for this purpose.

In the previous exercise this database was filled with data to demonstrate functioning of correctly created data warehouse. However, when creating a data warehouse you will not

happen to have a database preloaded with needed data. You will need to use an ETL process to fill data warehouse database with required data. Therefore the preloaded data needs to be cleared, before beginning the process (Step 1).

If you have not created the data warehouse for the bookstore organisation, you can still run the ETL process, but instead of clearing the data warehouse database (omit Step 1) you need to create a new empty one (use bookstore-schema.sql). Also you need to disable "Analysis Services Processing Task" in Data import.dtsx package, as you have no data warehouse to process.

1. Clear tables of the data warehouse using clear_bookstore.sql script.
2. For one time start-up open the "Initial Data.dtsx" package, set correct file locations and server name for all connection managers (update every item in Connection Manager window). For the connection manager localhost.auxiliary "Test Connection" will fail - the database does not yet exist and will be created as a part of the start-up process.
3. Select Start. All tasks should be successful. You can check data flow for "Inserting into Date" Data Flow Task. In the Microsoft SQL Server Management Studio, you can check that the Date, Time and Junk tables have been filled out.
4. Setting up data sources for T1 time: create a database for the computer system (BillMaster-schema.sql) and fill it with data to point T1 (1.BillMaster.sql) – run both scripts in SQL Management Studio.
5. Open the "Recurrent Loading.dtsx" package and set the correct server names or file locations for all connection managers (for "CEO's Excel File" Connection Manager select the "1.CEO's.xlsx" file). Note that "localhost.bookstoreDW2 analytical database" connector needs to be configured to connect to the bookstore database on Analysis Services Server, the name of the Initial catalog should be the same, as the name of your Multidimensional Project (if you have deployed it to the Server). Also make sure that in "Processing Bookstore Data Warehouse" component of the control flow you choose your data warehouse to process (example uses "BookstoreDW2" warehouse). Also, for the data warehouse to process correctly, if you use **specific Windows user name** for "Impersonation Information" in Data Source component of your Multidimensional Project, you **must also provide user password**. Additionally, when browsing for "Zip-code Listing CSV File", txt file extension will be selected by default and need to be changed to csv extension.

If you have not deployed the bookstore data warehouse, you should turn localhost.BookstoreCube connector offline and disable Analysis Services Processing Task, as noted in Step 0.

6. Select Start. The remaining tables should be filled out.
7. Setting data sources for T2 time: add data from time period T1-to-T2 to the sales system database by running "2.BillMaster.sql" script (do not delete previous data), change the file in "CEO's Excel File" Connection Manager from "1.CEO's.xlsx" to "2.CEO's.xlsx" (note, that the construction of this ETL process is such, that loading of T2 data can be done after loading T1 data – protections against adding the same data more than once are built into the process).
8. Select Start for "Recurrent Loading.dtsx" package again to start data import. Data from T1-T2 will be added.

A general review of the project:

Package "Initial Data.dtsx":

The "localhost.auxiliary" connection manager has {DelayValidation} property set to True, because the auxiliary database is created by the task "Creating auxiliary tables" and the manager cannot be connected to this database before it is created. Also tasks "Inserting into Holidays", "Inserting into School Breaks" and "Inserting into Date" have {DelayValidation} property set to True because they use "localhost.auxiliary" connection manager.

Green arrows in Control flow (initial view of the package) denote task precedence (task at the beginning of an arrow must be completed before task at the head of an arrow may start), in Data flow green arrows indicate data flow.

Roles of individual tasks:

“Creating Auxiliary Tables” – uses an SQL script to create database “auxiliary” and tables for holidays and school breaks. Those tables, after being filled with data in separate tasks” will be used to insert dates into the data warehouse.

“Inserting into Holidays”, “Inserting into School Breaks” - they fill “holidays” and “school_breaks” tables.

“Dropping Auxiliary Tables” - after use, the “holidays” and “school_breaks” tables are removed (as well as the entire “auxiliary” database).

All of these tasks are of the Execute SQL task type and use external files.

There are also two Execute SQL task tasks that use the SQL statements given directly. These are “Inserting into Junk” and “Inserting into Time” that fill “Junk” and “Time” tables respectively.

The last task (“Inserting into Date”) is of type Data Flow Task and is used to fill in the “Date” table. Double-clicking this task will take you to the Data Flow tab , where the data flow created for this task will be displayed.

Data flow of “Inserting into Date” task:

In order to be able to check freely the settings of individual components (in particular lookups), before inspecting the data flow, run the task “Creating Auxiliary Tables”. This will enable the connection manager “localhost.auxiliary” to perform “Test Connection” procedure - the program will not report errors when viewing column mappings. After reviewing the dataflow, you must delete the auxiliary tables (run “Dropping Auxiliary Tables” task).

The data flow begins with an OLE DB source called “Script generating dates”. It uses the SQL command to generate dates from the selected range (@Start = '1980-01-01', @ End = '2015-12-31'). In addition, additional fields are created based on the generated date, such as Year, Month, weekDay, etc..

Then, for a given date, it is determined whether it is a holiday - lookup called “Adding holidays”. The lookups searches the “holidays” table (“Connection” tab) for entries with the same date as considered. If found, the “holiday” and “bank_holiday” fields are added to the data flow (specified in the “Columns” tab). If not, the rows without the corresponding entries in the “holidays” table go to “No match output ”. In order to achieve this the default way of handling rows with no matching entries needed to be changed to “Redirect rows to no match output” in the “General” tab.

For holidays, it is checked whether they are bank holidays (“bank_holiday” column). If they are, then the value of the “workingDay” field is set to “day off”. If not, then the value set in “Script generating dates” (based on the day of week) is left unchanged – see the “Setting workingDay” component. For non-holiday days, a “holiday” field with the value “ordinary day” is added – see the component “Setting as 'ordinary day’”. Then both flows are combined (Union All component).

The next lookup (“Checking for holidays on the next day”) performs an analogous operation, but for the day before the holiday. Therefore, the source for the lookup is not the table, but the query which subtracts 1 from date column and adds “the next day was “ to the name of the holiday. In case no

holiday is found a field called "HolidayEve" is added with a value of "the next day was ordinary day" ("No holiday on the next day" component) and both data flows are combined.

The last lookup ("Adding school breaks") performs an operation to determine if the date is during a school vacation. The operation would be analogous to the previous ones, but here you have to check whether the checked date is between the beginning and the end of any holiday. To do this in lookup component, after selecting the "school_breaks" table as the source, the query used is customised. To use custom query "Cache mode" in the "General" tab must be set to "Partial cache". This enables "Modify the SQL statement" checkbox in the "Advanced" tab. Checking it and providing SQL query enables custom query. In this case the SQL statements looks for school break that started before the considered date but not yet ended (BETWEEN statement). The value of "Date" column will be substituted for the question mark in the SQL statement (press "Parameters" button to see the configuration; note that the Date column needs to be used in "Columns" tab to be available as a parameter in custom query).

Finally, the date is inserted in the warehouse table "Date" ("OLE DB Destination"). Please note that no value is entered for the surrogate key ("Mappings" tab). The "Date" table have been configured to automatically generate values for its surrogate key for each added row. Please accept this as a standard – providing a value for the surrogate key in the ETL process will be considered an error.

"Recurrent Loading.dtsx" package:

The package includes 7 data flow tasks . 2 of them concern inserting facts, 5 inserting dimensions. The green arrows set the precedence. In general, to insert facts, you must first insert all dimensions associated with them so that you can retrieve their surrogate key values to use as foreign key values in the fact table. (You have to retrieve them from the data warehouse, as they cannot be set to any value from the data sources - they will receive values in the ETL process that should be considered random). In this case, before introducing employees, bookstores should be inserted - the employee will be assigned to the bookstore, and the employee's insertion is broken down into two stages (enclosed in Sequence Container "Populating Seller Table") due to the recursive hierarchy. Sequence Container is used to distinguish all tasks involved in adding sellers to the data warehouse. As an additional benefit it allows for running all the task in a container instead of running each of them separately or all of the tasks.

In the "Populating Bookstore Table" task :

Information on bookstores is downloaded from sheet 1 of the excel file ("Bookstores sheet"). There is no data on provinces in them - these are downloaded from the "locations.csv" file ("Zip-code listing"). The data is joined by the zip code - hence the sorting of both data flows by this column. For the data from the csv file, zip code repetitions are rejected - it was assumed that all places with the same postal code are in one province (if there are 2 entries with the same code and different provinces, then random ones will be selected).

For a bookstore, its size is needed, which is determined by the number of employees. Hence, the employees are taken from sheet 2 of the CEO's excel ("Employees Sheet"); employees who no longer work are rejected ("Filtering out past employees") and the rest is counted relative to the bookstore identification number (Aggregate type component – "Counting employees"). The size of the bookstore is determined on the basis of the count (Derived Column component – "Bookstore size") and then the main data on the bookstore is combined with the size. The bookstore is treated as a slowly changing dimension - the size of the bookstore is changing , and the new size replaces the old one - the Slowly Changing Dimension component has been set appropriately (all non-key columns are set as changing attributes). All components following Slowly Changing Dimension component are added automatically after its configuration is finished.

Three remarks here:

- Data read from Excel files is always in Unicode, so it needs to be converted to “ANSI – Latin I” used in the data warehouse. Also numbers are represented as floating point numbers, so bookstores’ ids need conversion too. To check output format of excel files use “Show Advanced Editor...” from Excel source context menu (available for most components).
- Slowly Changing Dimension component allows for handling SCDs type 1 and 2 and a mix of the two types. It checks each incoming row whether the business key is already in the data warehouse or not. If not, such row is added to the data warehouse. If the business key is already in the data warehouse the components checks whether any other columns are changed. If historical columns are changed, the old value is marked as inactive and the row with new value is inserted into data warehouse. If changing columns are changed, the data in the data warehouse is updated to reflect the new values. If fixed column is changed, an error is raised. Rows with no changed columns are ignored. Type of each column in the table is decided separately and can differ from other columns.
- Merge Join component (such as “Assigning province to bookstores”) merges two data flows by adding together selected columns from each of the flows based on a matching value of one or more columns used as the join key. You can use INNER, LEFT OUTER or FULL OUTER JOIN types. Merge Join component requires both input data flows to be sorted according to the join key (or sorted with a sort key prefix being the join key).

“Populating Seller Table”

There are a few peculiarities about the seller dimension:

- It is a slowly changing dimension,
- It has a recursive hierarchy (foreign key referencing itself),
- Data source for this hierarchy is an Excel file,
- There can be many entries for a single seller in the file - some changes to dimension members are denoted through adding new entries to the file (change of position),
- Some changes to dimension members are denoted by updating ALL entries about these members in the file.

Some of this characteristics are fairly frequent, but combining them together results in a more complicated data load process:

1. Because the dimension includes self-reference, surrogate keys for each seller must be generated before inserting sellers’ data (will be handled by using inferred members mechanism) or data transfer must be split into parts according to the number of hierarchy levels (not done in this case).
2. Because there could be more than one entry for each seller, old entries must be filtered out in order for SCD processing to function properly.
3. And yet, because some changes are denoted by updating seller data (including old entries) some old entries in the source file with newer data might differ from the same entries in the older file, and therefore from the data in the data warehouse, which complicates filtering out of old entries.
4. And finally, since the data source is a file, custom SQL queries cannot be used to extract the data.

The solution to those problems are:

1. There is an initial data load that inserts only personal ID numbers (and IsCurrent flags) for new sellers to facilitate data loading and a main data load that uses inferred members support of Slowly Changing Dimension component to add missing information to such entries (as well as properly updating changed information).
2. The main data load filters out rows of data already present in the data warehouse (data loaded from the file is compared not only with current data warehouse entries but also dated ones resulting in filtering out current data already present in data warehouse but also old entries).
3. To facilitate filtering out updated old entries, changed names and education levels are updated in data warehouse in the initial data load (before the main data load begins). This results in education attribute of the dimension to be SCD type 1 rather than type 2, that would be a more optimal choice, but would also result in a more complicated process.
4. Another simplification is assigning bosses based on bookstores' directors at the time of load. This will work fine for recurrent loads, but for the first loading will assign one boss to every salesperson independent of how many directors their bookstore had to date. Again, time of employment of each director and salesperson could be taken into account to calculate overlapping time periods and finding all of historic bosses for each salesperson, but that would complicate the loading process, especially because this time periods matching cannot be done on data source. This simplistic approach to determining bosses would also result in assigning new bosses to past employees, and to avoid that while filtering out old data in the main data load also entries with new bosses (as determined by the load process) but otherwise identical to those in the warehouse are filtered out for salespeople no longer employed by the organisation.
5. An assumption was also made that there cannot be more than one new entry in the Excel file for each employee per data load. This is supported by the example data.

The resulting two data load processes described further are enclosed in a Sequence Container "Populating Seller Table" to group them together. Using Sequence Container has also the added benefit of enabling these two processes to be executed together (one can execute a whole SSIS package, a single task or tasks held in a single container).

"Initiating Employee Load" data flow

This data flow retrieves all of the sellers personal ID numbers, names, surnames and education from the source Excel file ("Employees Sheet"). After needed conversions are done ("Conversion from Unicode") full name is calculated ("Calculating FullName"). Then values with repeating PIDs are removed using Sort component and all columns are filtered out apart from "PID", "Education" and "NameAndSurname" ("Remove duplicates" option) – as, whenever name or education is changed, all entries are updated, the removed values should be identical to the ones with matching PID that are passed through. Next old (and possibly changed entries) are separated from new entries using Lookup component "Split old and new entries": for every row on input it checks whether given PID is already in the data warehouse or not (note that rows with no matching entries are redirected to no match output {not a standard behaviour}). To new rows ("Lookup No Match Output") a "current" flag with value "TRUE" is added ("Adding isCurrent Flag" component) and they are added to the data warehouse ("Seller (DW)" component). Old/updated rows ("Lookup Match Output") are passed to a Slowly Changing Dimension that will ignore rows with education/name values matching those in the

data warehouse and update the warehouse in case a change is detected there. The configuration is: "PID" set as a business key, "Education" and "NameAndSurname" set as changing attributes and update changed attributes even in outdated entries is enabled.

Running this task results in adding surrogate key, PID and "IsCurrent" flag with all other columns of the "Seller" table set to "null" for every new employee (PID not present in "Seller" table) and in updating full name and education in every relevant row of "Seller" table for employees that had them changed. The empty columns will be set in the next task, while earlier generation of surrogate keys values allows for them to be used in the "Boss" self-referencing foreign key.

"Loading Employees" data flow

Data flow for loading employees is a bit more complicated. It starts with loading employee data for Excel file ("Employees Sheet" component) and doing necessary data format conversions ("Conversion from Unicode" component). Next from the loaded data age, experience and full name are calculated in a Derived Column component ("Calculating Age, Experience and FullName"). Age and experience is calculated as a difference in years between a birthday or an employment date respectively and a termination date if available. If termination date is not available, current date is used. Full name is a simple concatenation of name and surname with a space added in between. Than age and work experience are divided into ranges by using a Script Component ("Calculating ranges"). Script component is used because the formulas required for Derived Column component would be complex.

The next part of the data flow is used to determine Boss foreign key values. Though in general, as it is a recursive reference, there could be arbitrary number of boss hierarchy layers, in this particular case there are only two. The rule in this case is simple: each bookstore has one director, who is the boss of all salespeople from his/her bookstore. Directors have no bosses. To achieve this all of the employees are split into sellers, who will have their bosses set, and directors, who will not. Directors are further divided into current directors (who will be used to determine sellers' bosses) and old directors (by Conditional Split component called "Finding Bosses"). Current directors data flow is duplicated by Multicast component.

The concurrent data flows are:

- Director 1 ("Old directors"): Both current and past directors are combined in this flow using "Union All 1" component (used for unsorted data flows or when the sorting does not need to be preserved) and sorted using Bookstore ID ("Sort by bookstore ID (3)" component). This ordering will be useful later in the data flow. Note that Bookstore ID is so far a value from the source database, so it cannot be used in data warehouse. Next ("Adding empty boss surrogate key to bosses") a column "Boss DW ID" with a value of "NULL" is added, as directors have no bosses.
- Director 2 ("Directors"): This flow is used to determine sellers' bosses. It starts with sorting the data according to Bookstore ID ("Sort by bookstore ID (2)" component). During sorting rows with duplicate Bookstore ID are removed as a precaution (this should not be needed as, as a rule, each bookstore has only one director) and only "PID" and "Bookstore ID" are passed further. Than a "current" flag with a value of "TRUE" is added to each row. This is done so that in the next step we can look for each of the directors current surrogate key

based on the "PID" and "isCurrent" columns. This is done in "Finding bosses' Surrogate Key" component, where for each director his/her surrogate key is retrieved from data warehouse's "Seller" table and added to the data flow as "Seller DW ID" column. Then this flow is merge joined with "Seller" data flow.

- Seller: First for each seller a position name is changed from "salesperson" used in the data source (Excel file) to "seller", as used in the data warehouse ("Updating sellers' position name" component). Then the data flow is sorted according to "Bookstore ID", this time rows with repeating "Bookstore IDs" are not removed, as there will be many sellers in each bookstore ("Sort by bookstore ID"). The ordering is immediately used in the next component: Merge Join called "Adding bosses' surrogate keys to sellers" that adds bookstore's director surrogate key (taken from "Director 2" data flow) to the information about seller based on the value of "Bookstore ID" (join key). Then rows from "Seller" and "Director 1" data flow are combined into a single "main" dataflow using Merge component called "Merging all employees". Merge components do UNION operation on input data flows but, in contrast to Union All components, they preserve the order of data in data flow (therefore all input data flows must be sorted using the same sort key).

Next part of the data flow is determining the surrogate key of the employees' bookstores of employment. In the "main" data flow there is a "Bookstore ID", but it is not used in the data warehouse, so it cannot be used. Instead bookstore name will be used, and this makes a new data flow required starting with "Bookstores sheet" components that loads information about bookstores from Excel file. Then, after required data format conversions ("Converting from Unicode" component), the data is ordered according to "Bookstore ID" with removal of duplicate entries ("Sort by bookstore ID (4)"). Then for each bookstore its data warehouse surrogate key is retrieved based on its name (business key) using Lookup component ("Finding bookstores' surrogate keys").

Resulting data is filtered before being passed to Slowly Changing Dimension component (because of multiple entries per person). First the data stream is split into current employees and old employees ("Separating Old Employees"), then it is filtered using Lookup components ("Removing old entries" and "Removing old entries (2)") and combined back together ("Union All" component). The filtering differs from the filtering done by the Slowly Changing Dimension component in that it considers also expired entries when filtering data. Filtering done for old employees also disregards any changes to boss attribute as per discussion in the introduction to this dimension. Filtering out old entries works correctly even if an entry have been updated owing to updating of employees full names and education done in "Initiating Employee Load" task.

Filtered information is passed to a Slowly Changing Dimension component. The configuration is as follows: "PID" is a business key, "NameAndSurname", "Education" and "ID_Bookstore" are fixed attributes (full name and education is updated by previous task, so no new values should be found), current and expired records are marked by a single column ("IsCurrent") and all remaining attributes are historical. Inferred member support is enabled (with inferred members recognised by having all change columns set to "NULL"), which means that the component will add all the missing information to "Seller" table entries added in "Initiating Employee Load" data flow. All components following the Slowly Changing Dimension component are generated by it.

Note: Inferred member support is a mechanism by which a Slowly Changing Dimension component distinguishes between missing information and changed information. If a row in a data warehouse table is determined to be missing information and new values are provided, such row is updated regardless of attribute change types (fixed or historical attributes will be updated). Missing information in a row can be indicated by a Boolean column with the value set to "TRUE" or by setting all of the changing columns' values to "NULL" (as is in this case; note that changing columns in this case mean changing attributes, historical attributes and fixed attributes).

Note also: Script Component enables you to use .NET code when transferring data into data warehouse. There are three types of script components:

- Source – can be used to produce data to be used in ETL process. It has no input flow, only output flows.
- Destination – the opposite of Source type. Can be used to store produced data in a custom way. It has an input flow, but no output flow.
- Transformation – can be used to change values of input flow columns or add new columns to the data flow. Has both input flow and output flows.

When configuring a Script Component you need to decide to which columns from the input flow (if supported for a given Script Component type) you want to have access in your .NET script and the type of access you need ("Input Columns" tab). You also need to specify output columns (again, if supported) for your output data flow and possibly add another output data flow if needed ("Inputs and Outputs" tab). In case of Transformation script output columns will be added to columns on input. For each Script Component type you can also add connection managers you plan to use ("Connection Managers" tab). The script itself can be edited by pressing "Edit Script..." button in the "Script" tab.

"Populating Book Table" data flow

Inserting data about books is quite simple - the information about the book ("Book (BillMaster)" component) and its sale ("BookSale (BillMaster)" component) is taken from the source. On the basis of sales information, price ranges in which a book with a specific ISBN was sold are determined ("Calculating PriceRange" component). Both data flows are sorted using ISBN and in case of sales data also by "PriceRange" to remove duplicate {ISBN, price range} pairs (components "Sort by ISBN" and "Sort by ISBN and PriceRange"). After merging the flows using ISBN as join key ("Adding price range" component), the information already in the warehouse is filtered out - information about books with the same ISBN number and price range (two part business key) is searched for in the data warehouse ("Dropping old values" components), only rows for which no existing entry was found in the warehouse are added to it ("Book (DW)" component). As always, when inserting into dimension table, the surrogate key is not set.

Note that in order to filter out values already present in the data warehouse Lookup component was used. The table that was searched was the data warehouse destination table ("Book") and the matching was done by ISBN and price range. Then "Lookup no match output" was used as the source of data to be loaded into the data warehouse (component configured to redirect rows with no matching entries to no match output). This construction results in only those {ISBN, price range} pairs that are not present in the "Book" table to be added to it. This procedure is typical for loading data

into the data warehouse with the exception of slowly changing dimensions (Slowly Changing Dimension component filters out old values as part of its functionality).

“Populating Author Table” data flow

Adding information about authors is very straightforward. Data is taken from the source (“Author (BillMaster)” component), full name is produced based on first name, second name (if present) and surname (“Calculating Full Name” component), data already in the data warehouse is filtered out (“Dropping old values” component) and remaining values are added to data warehouse’s “Author” table (“Author (DW)” component).

Inserting into Authorships

Introducing facts in general consists in retrieving information about the fact from the source and about all dimensions with which it is associated. Then the appropriate surrogate keys from the dimension tables are retrieved based on the information about the associated dimensions. It may also be necessary to calculate the measure values if they are not given explicitly in the source.

In the case of authorship data from the many-to-many “Authorship” table (“Authorship (BillMaster)”) and data about authors (“Author (BillMaster)” component) are retrieved from the source database and joined using “Author ID” (“Merging Author IDs and Books” component). Then authors full name is produced (“Calculating Full Name”) and author surrogate key is retrieved (“Retrieving Author DW ID”). Next, books surrogate keys and ISBN numbers are retrieved from data warehouse (“Book (DW)” component) and joined using ISBN number with authorship data (“Combining Author DW ID with all Book DW IDs” component) creating pairs of authors’ and books’ surrogate keys. Next information already present in the data warehouse is filtered out (“Dropping old values” component) and remaining information is added to the warehouse (“Authorship (DW)” component).

“Populating Book_sale Table” data flow

Inserting sales facts proceeds in the same way. Sales data is collected (“BookSale (BillMaster)” component), price category is calculated for each row (“Calculating PriceRange”) and then the data is combined with receipts (“Bill (BillMaster)” component). In “Derived Columns” component additional data processing is done. It includes calculating purchase price of the books or extracting hour of purchase from DateTime column among others. Next surrogate keys for all related dimensions are retrieved from the warehouse (components: “Retrieving Book DW ID”, “Retrieving Time DW ID”, “Retrieving Junk DW ID”, “Retrieving Sale Date DW ID”, “Retrieving Payment Date DW ID”, “Retrieving Seller DW ID”). For the seller, a surrogate key is retrieved from the current entry (“IsCurrent” == “TRUE”). As usual, before inserting the data is checked whether it already is in the warehouse (“Dropping old values” component). When inserting sale data (“BookSale DW” component), “Profit” column is not set as it will be calculated automatically by the database (it is defined as a calculated column in the “bookstoreDW.sql” script).

“Processing Bookstore Data Warehouse” Task

The last step is to process the inserted data so that it can be loaded into the Analysis Services Server and become visible in the data warehouse. It is achieved by using an Analysis Services Processing Task configured to connect to target cube in the Analysis Services Server. The scope of the processing

is set in “Processing Settings” tab and full processing of the whole database should work in most cases.